

6. INGENIERÍA DE DESPLIEGUE EMBEBIDO

En esta fase.

[Completar – Introducción al capítulo – Mención breve de cómo se organiza o compone]

6.1.Arquitectura de Implementación y Despliegue en ESP32-S3

En esta sección se detalla la configuración técnica y la infraestructura de software diseñada para permitir la ejecución de modelos de aprendizaje profundo en el SoC ESP32-S3. A diferencia de los despliegues convencionales, esta arquitectura se fundamenta en la optimización de memoria y el uso de librerías nativas para maximizar el rendimiento del hardware.

6.1.1. Configuración del SoC

Como ya fue detallado en la sección 4.3, el proyecto utiliza el módulo ESP32-S3 WROOM-1 (Freenove), cuya arquitectura de doble núcleo se configuró para operar a su frecuencia máxima de 240 MHz. Un componente vital para el manejo de tensores de visión es la PSRAM Octal de 8 MB, configurada a 80 MHz para maximizar el ancho de banda.

La adquisición de datos visuales se realiza mediante el sensor OV5640. A diferencia de las configuraciones estándar que utilizan JPEG, este proyecto implementa el formato RGB565 (2 bytes por píxel) para evitar artefactos de compresión y facilitar el preprocesamiento directo en memoria.

- **Configuración de Memoria:** Se implementó un double buffering directamente en la PSRAM para evitar la pérdida de cuadros durante los ciclos de inferencia.
- **Ajustes del Sensor:** Se aplicó un giro vertical (vflip) por hardware y se activaron los controles automáticos de balance de blancos, ganancia y exposición para asegurar la consistencia lumínica en entornos domésticos.
- **Resolución:** La captura nativa se estableció en QVGA (320×240 píxeles), lo que representa una carga inicial de 153,600 bytes por cuadro. Que luego es reescalado a 224x224 pixeles por corte de bordes con offset.

Figura 52. Estructura del Proyecto

```

03_ING_DESPLIEGUE/
├─ CMakeLists.txt          # Top-level ESP-IDF build (proyecto activo)
├─ sdkconfig.defaults      # Configuración base ESP-IDF (Ciclo 3)
├─ partitions.csv         # Tabla de particiones flash (Ciclo 3)
├─ requirements.txt        # Dependencias Python (esp-ppq, torch, tf, onnx...)
├─
├─ main/                  # — Componente principal del firmware —
│   ├── app_config.h       # Configuración global (pines, resoluciones, umbrales)
│   ├── main.cpp           # Entry point: orchestrator (init + inference loop)
│   ├── camera.cpp/.h      # Driver OV5640 (RGB565, QVGA, double buffer)
│   ├── image_proc.cpp/.h  # Preprocesamiento: RGB565→INT8 (TFLite/ESPDL)
│   ├── inference_engine.h  # Interfaz abstracta (virtual) para motores
│   ├── tflite_engine.cpp/.h # Implementación TFLite Micro (26 ops registrados)
│   ├── espdl_engine.cpp/.h # Implementación ESP-DL v3.x (mmap desde partición)
│   ├── postprocess.cpp/.h  # Decodificación: MBNTv2/YOLO11/YOLO26/FCOS/DFL
│   ├── metrics.cpp/.h     # Métricas EMA + sensor de temperatura
│   ├── wifi_manager.cpp/.h # WiFi STA (conexión a red doméstica)
│   ├── web_server.cpp/.h  # HTTP + WebSocket + MJPEG en puertos 80/81
│   ├── stream_buf.cpp/.h  # Buffer thread-safe JPEG (producer-consumer)
│   ├── dashboard.h        # HTML gzip'd auto-generado
│   ├── CMakeLists.txt     # Registro de componente (C++23, REQUIRES)
│   ├── idf_component.yml  # Dependencias: esp32-camera, esp-tflite-micro, esp-dl
│   └─ frontend/
│       ├── dashboard.html # Dashboard interactivo (MJPEG + canvas bbox + WS)
│       └─ models/tflite/  # Modelos TFLite embebidos como arrays C (legacy)
│   └─
├─ models/                # — Modelos entrenados —
│   └─ espdl/              # Modelos ESPDL producción (.espdl + .info + .json)
│   └─
├─ scripts/               # — Scripts de soporte —
│   ├── convert_onnx_to_espdl.py # Conversión ONNX→ESPDL via esp-ppq
│   ├── flash_models.sh         # Flasheo independiente de modelos
│   ├── gen_dashboard_header.py # HTML-gzip-C header
│   ├── validate_partitions.py  # Validación de tabla de particiones
│   └─ ...                      # inspect_tflite, list_ops, etc.
│   └─
├─ deployment_cycles/     # — Documentación por ciclo iterativo —
│   ├── README_cycle1.md   # Ciclo 1: Despliegue base (3 modelos TFLite/ESPDL)
│   ├── README_cycle2.md   # Ciclo 2: Diagnóstico cuantización YOLO
│   └─ README_cycle3.md    # Ciclo 3: Integración final ESPDL
│   └─
├─ docs/                  # — Instructivos técnicos —
│   ├── Configuración_ESP32-S3.md
│   ├── Instructivo_Despliegue_ESPDL.md
│   └─ Instructivo_Despliegue_YOLO26_T3.md
│   └─
├─ outputs/               # — Capturas de inferencia on-device —
│   ├── Inference_Embedded_1.txt # Frame JPEG capturado desde el dashboard
│   ├── Inference_Embedded_2.txt
│   └─ Inference_Embedded_3.txt
│   └─
└─ components/            # — Componentes ESP-IDF locales —
    └─ espressif_esp-tflite-micro/

```

El desarrollo se basó en el *framework* **ESP-IDF**, utilizando el estándar de lenguaje **GNU++23** (*-std=gnu++2b*) para aprovechar las optimizaciones modernas de C++ en la gestión de estructuras de datos. Para la gestión de red, se aplicaron optimizaciones de velocidad en la IRAM para el stack WiFi y se incrementó el límite de sockets abiertos a 10, asegurando que el servidor de telemetría y el flujo de video no colapsen durante la transmisión de datos. Finalmente, la integración del componente esp32-camera se realizó mediante vinculación directa en el sistema de construcción *CMake*, garantizando el acceso a las funciones de bajo nivel del sensor.

6.1.2. Gestión de Recursos

La gestión de memoria fue configurada mediante el **SDK ESP-IDF v5.4.3**, estableciendo parámetros específicos en el *menuconfig* para habilitar el acceso a la PSRAM mediante *heap_caps_malloc*, lo cual resulta imprescindible para alojar tanto los tensores de activación de los modelos como los búferes de la cámara. Debido a la alta demanda de memoria interna tras la inicialización del *stack* de *WiFi* y el motor de IA, se optó por alojar el *stack* de la tarea de inferencia directamente en la PSRAM utilizando *xTaskCreatePinnedToCoreWithCaps*, evitando así errores de desbordamiento en la SRAM interna.

Se diseñó un esquema de almacenamiento personalizado para optimizar el uso de los 16 MB de memoria Flash del dispositivo. En lugar de embeber los modelos dentro del binario del firmware (método *MBED_FILES*), se implementó un sistema de mapeo por particiones independientes definido en el archivo *partitions.csv* detallado en la Tabla 21. Este enfoque permite realizar actualizaciones de los modelos de detección mediante *esptool.py* de forma independiente al firmware, facilitando procesos de iteración y validación sin necesidad de recompilar el proyecto completo.

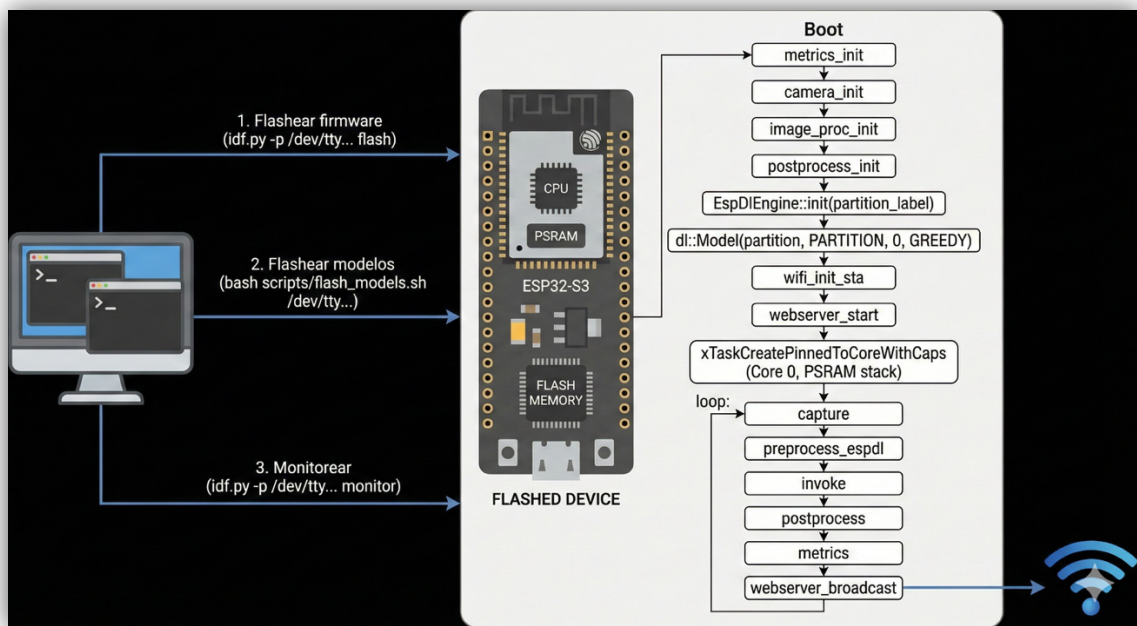
Tabla 21. Gestión de memoria Flash a través de *partitions.csv*

Partición	Tipo	Subtipo	Tamaño
Factory	App	Factory	10 MB
Model_espdet	Data	0x40	1 MB
Model_yolo26	Data	0x40	3 MB

6.1.3. Orquestación del Firmware y Motor de Inferencia ESP-DL

El corazón del sistema reside en la integración de la librería **esp-dl v3.x**, una biblioteca de alto rendimiento optimizada para las instrucciones SIMD (*Single Instruction Multiple Data*) del ESP32-S3. El flujo de ejecución, ver Figura 53, se distribuyó de forma asíncrona entre los núcleos del SoC: el Núcleo 0 se dedica exclusivamente a la captura, preprocesamiento e inferencia, mientras que el Núcleo 1 gestiona la pila de red y el servidor web donde se visualiza el streaming y métricas.

Figura 53. Esquema del Flujo de Despliegue



Uno de los mayores retos técnicos resueltos fue la estandarización del preprocesamiento para modelos INT8. Se implementó una función de normalización, basada en la ecuación 5, que convierte los píxeles de la cámara (RGB565) a un rango compatible con la cuantización simétrica del motor ESP-DL. Esta corrección fue importante para asegurar la coherencia entre los resultados obtenidos en la fase de entrenamiento y el rendimiento *on-device*, eliminando errores de detección causados por desajustes en la escala de los tensores de entrada.

$$val_{int8} = clamp\left(round\left(\frac{pixel \times 128}{255}\right), 0, 127\right) \quad \text{Ecuación (5)}$$

6.2. Sistema de Monitoreo y Visualización de Detecciones

Para validar objetivamente la capacidad de percepción del prototipo LCMR y recolectar métricas de desempeño en tiempo real, se desarrolló una infraestructura de telemetría basada en servicios web embebidos. Esta herramienta permite la supervisión remota del estado del sistema y la validación visual de las inferencias realizadas por los modelos de aprendizaje profundo. En la Figura 53 se muestra el dashboard *front-end* diseñado.

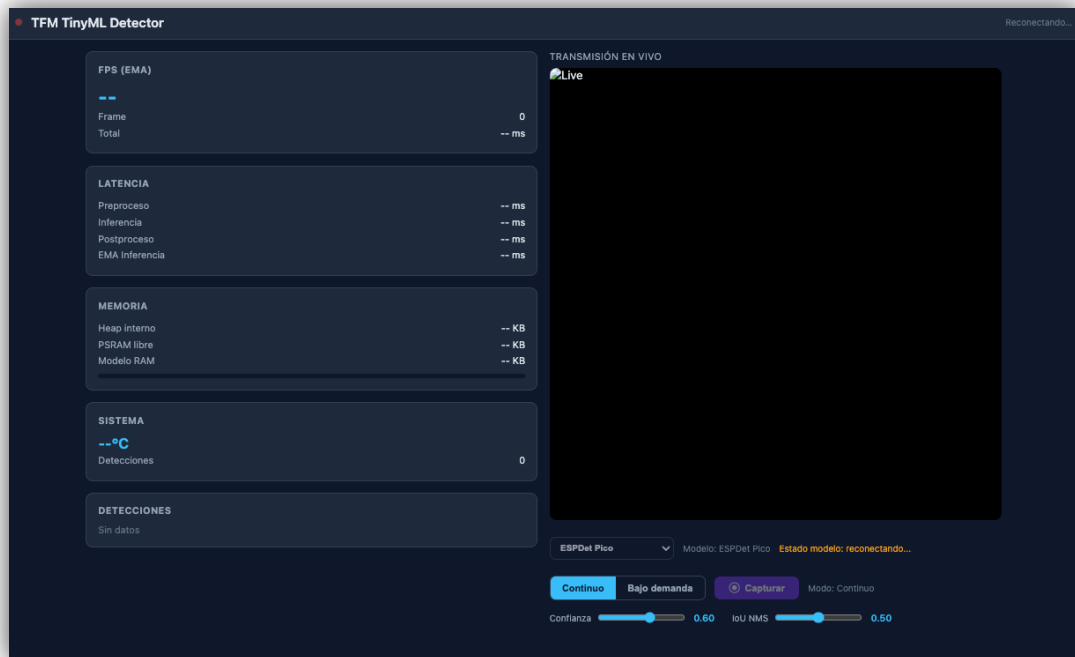
6.2.1. Arquitectura del Dashboard

La interfaz de visualización se implementó como una aplicación web de una sola página (SPA) alojada en la memoria *flash* del SoC, utilizando compresión *gzip* para optimizar el espacio de almacenamiento. El sistema de comunicación se fundamenta en un servidor HTTP que gestiona dos canales de datos paralelos:

- **Transmisión de Video (Streaming):** Se utiliza el protocolo MJPEG (*Motion JPEG*) a través del punto de acceso */stream*. Este flujo envía tramas de video comprimidas en tiempo real, desacopladas del ciclo de inferencia mediante un búfer compartido en la PSRAM para mantener la fluidez visual.
- **Canal de Control y Datos (WebSockets):** Se estableció un túnel bidireccional mediante *WebSockets* para la transmisión de metadatos en formato JSON. Este canal permite recibir comandos de configuración desde el usuario y enviar telemetría detallada del hardware y resultados de detección.

6.2.2. Visualización Dinámica y Procesamiento en Cliente

Con el fin de minimizar la carga computacional del microcontrolador, se delegó la lógica de renderizado de las cajas delimitadoras (bounding boxes) al navegador del cliente. El firmware transmite únicamente las coordenadas normalizadas $[x_1, y_1, x_2, y_2]$ y el identificador de clase correspondiente.

Figura 54. Sistema de Monitoreo y Visualización de Detecciones

El panel de control renderiza dinámicamente capas de CSS posicionadas absolutamente sobre el lienzo de video, aplicando códigos de colores específicos según la categoría detectada (por ejemplo, verde para personas, ámbar para obstáculos y rosa para escaleras). Esta técnica permite una visualización fluida sin añadir latencia significativa al proceso de inferencia en el ESP32-S3.

6.2.3. Modos de Operación y Captura de Métricas

El sistema implementa un esquema de monitoreo dual que adapta el comportamiento del robot según los requerimientos de la validación técnica:

- **Modo Continuo:** Ejecuta un ciclo ininterrumpido de captura, inferencia y transmisión de métricas, esencial para la navegación autónoma en tiempo real.
- **Modo Bajo Demanda (On-Demand):** Permite realizar capturas puntuales de alta resolución para análisis forense de la detección. En este modo, el sistema envía la imagen procesada como un bloque binario a través de *WebSocket*, garantizando la integridad de los datos sin el sobrecoste de codificación en *Base64*.

La infraestructura recolecta sistemáticamente métricas de latencia segmentadas por etapas (preprocesamiento, inferencia y postprocesamiento) y monitoriza el estado de la memoria

6.3. Análisis Comparativo de Desempeño

A continuación, se presentan los resultados obtenidos tras la validación on-device de los modelos seleccionados, evaluando su comportamiento en condiciones de carga real dentro del SoC ESP32-S3. El análisis se centra en la eficiencia temporal, el consumo de memoria y la estabilidad del sistema, como lo muestra la Tabla 22, comparando la implementación base con las optimizaciones estructurales propuestas.

Tabla 22. *Resultados On-Device*

Métrica	ESPDet-Pico T4	Yolo26N T3 ESP
Inferencia	~405 ms	~2885 ms
Preprocesamiento	~14-275 ms	~15-270 ms
Postprocesamiento	~0.7-1.4 ms	~3.5-4.8 ms
Total por frame	~550-680 ms	~3140-3170 ms
FPS	~1.5-1.8	~0.3-0.5
Detecciones/frame	1-3 típico	0-1 típico
Heap interno libre	~69 KB	~43 KB
PSRAM libre	~6288 KB	~3947 KB
Consumo modelo (PSRAM)	~1900 KB	~4240 KB
Parámetros modelo	361K	2.6M
Tamaño .espdl	545 KB	2.71 MB

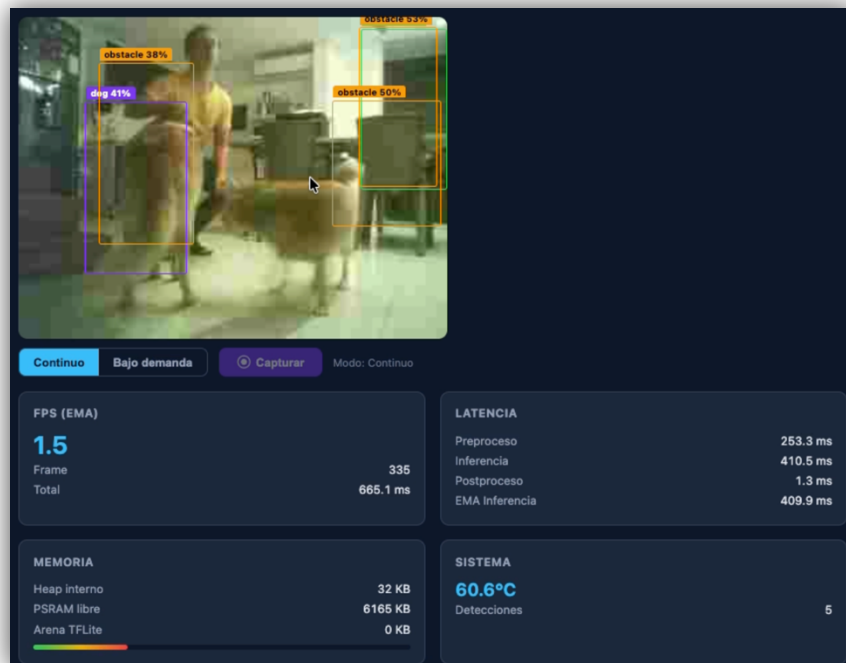
La evaluación del desempeño temporal permitió identificar una diferencia notable entre las arquitecturas evaluadas. El modelo ESPDet Pico T4, debido a su diseño liviano de 361K parámetros, logró una latencia de inferencia de aproximadamente 405 ms, lo que se traduce en una tasa efectiva de entre 1.5 y 1.8 FPS. Por el contrario, el YOLO26n T3 ESP, con 2.6M de parámetros, presentó una inferencia significativamente más lenta de 2,885 ms, resultando en una tasa de apenas 0.3 FPS, lo cual limita su aplicación para navegación en tiempo real.

El análisis de memoria demostró que ambos modelos son estables y no presentan fugas de memoria (*memory leaks*) entre *frames*, manteniendo niveles constantes de *heap* y PSRAM. Sin embargo, la huella de memoria difiere notablemente: mientras que ESPDet consume

~1,900 KB de PSRAM para activaciones, el modelo YOLO26n T3 demanda ~4,240 KB, dejando un margen corto de solo 43 KB de *heap* interno libre tras la carga.

A nivel técnico de la operación del hardware, durante las pruebas, se observaron advertencias del controlador de la cámara (*cam_hal: EV-VSYNC-OVF*) cuando el tiempo de inferencia superaba los 400 ms. Este fenómeno ocurre porque el *bus DMA* pierde sincronía al bloquearse el núcleo de procesamiento por tiempos prolongados; no obstante, el firmware implementado gestiona automáticamente el descarte de frames corruptos y la recaptura de nuevas tramas, garantizando la continuidad operativa del sistema sin degradación funciona.

Figura 55. *Pruebas de campo 1 Sistema de Detección de Objetos – Modelo ESPDet Pico T4*



Los datos cuantitativos confirman que el modelo ESPDet Pico T4 es la arquitectura más viable para el prototipo LCMR, cumpliendo con el requisito de detección consistente (1–3 objetos por frame) y manteniendo una latencia que permite la toma de decisiones en entornos dinámicos. Sin embargo, la evaluación cualitativa, de las cuales se muestran ejemplos en las Figuras 55 a la 59, en condiciones de operación real reveló discrepancias significativas en la precisión de sus predicciones.

The screenshot displays the TFM TinyML Detector application interface. The interface is dark-themed and shows real-time performance metrics and a video feed.

Left Sidebar Metrics:

- FPS (EMA):** 1.5 (Frame: 260, Total: 666.0 ms)
- Latencia:**
 - Procesado: 255.7 ms
 - Inferencia: 409.2 ms
 - Postproceso: 1.1 ms
 - EMA Inferencia: 407.0 ms
- Memoria:**
 - Heap interno: 27 KB
 - PIRAM libre: 5941 KB
 - Modelo RAM: 763 KB
- Sistema:** 62.6°C (Detecciones: 1)
- Detecciones:** 49g 75%

Main Area:

- TRANSMISIÓN EN VIVO:** A video feed showing a person's legs. A bounding box is drawn around the legs, and the confidence score is 75%.
- Controls:** Below the video feed, there are controls for the model (ESPOlet Pico), a dropdown menu, and buttons for "Continuar", "Bajo demanda", "Capturar", and "Modo Continuo".
- Confidence and FPS:** A slider for confidence is set to 0.70, and the FPS is 1.5.

Log Window (Bottom):

The log window shows detailed performance data for various models and hardware components. It includes timestamps, model names, and performance metrics like FPS and latency.

Uno de los hallazgos más críticos es el marcado sesgo categórico del modelo. A nivel de inferencia on-device, el sistema demostró una incapacidad consistente para detectar los objetos de tipo puerta (*door*) y escalera (*stair*). Las detecciones con alta confianza se limitaron predominantemente a tres clases: obstáculos (*obstacle*), perro (*dog*) y persona (*person*).

113

promedio (mAP) para las clases omitidas, reducir el error de regresión de las coordenadas de las cajas delimitadoras (Bounding Boxes) y aumentar la sensibilidad (recall) del modelo sin degradar la especificidad, evitando un incremento en la tasa de falsos positivos que pueda inducir a errores de navegación.

Figura 57. Pruebas de campo 3 Sistema de Detección de Objetos – Modelo ESPDet Pico T4

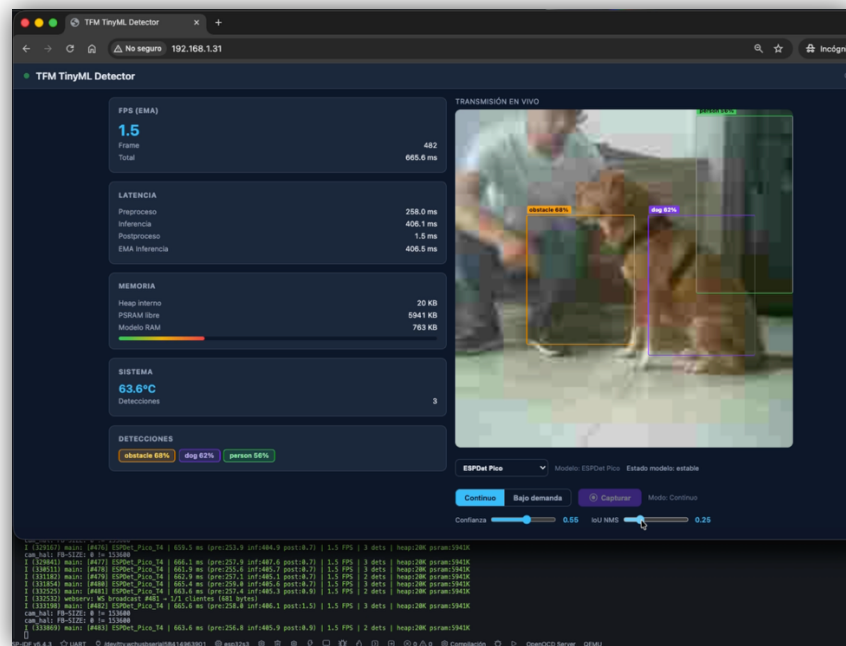


Figura 58. Pruebas de campo 4 Sistema de Detección de Objetos – Modelo ESPDet Pico T4

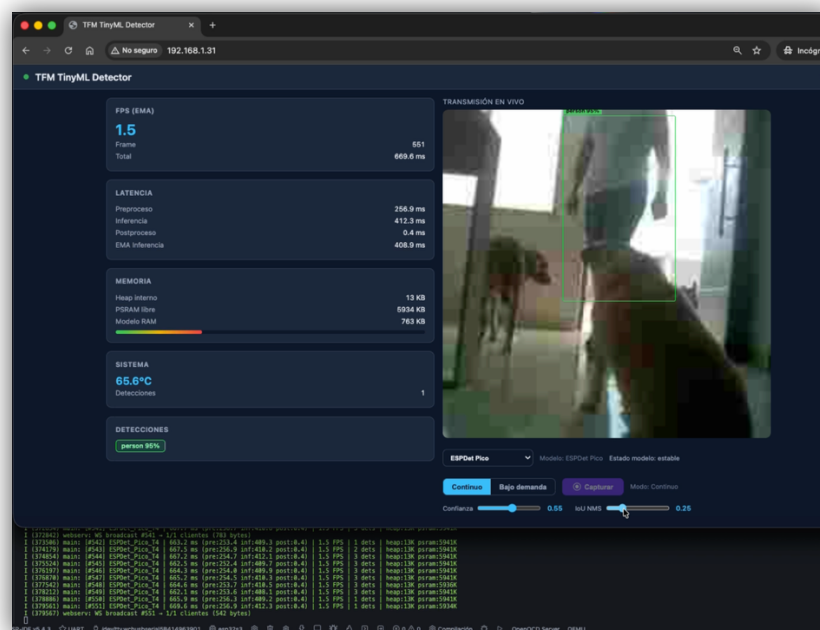
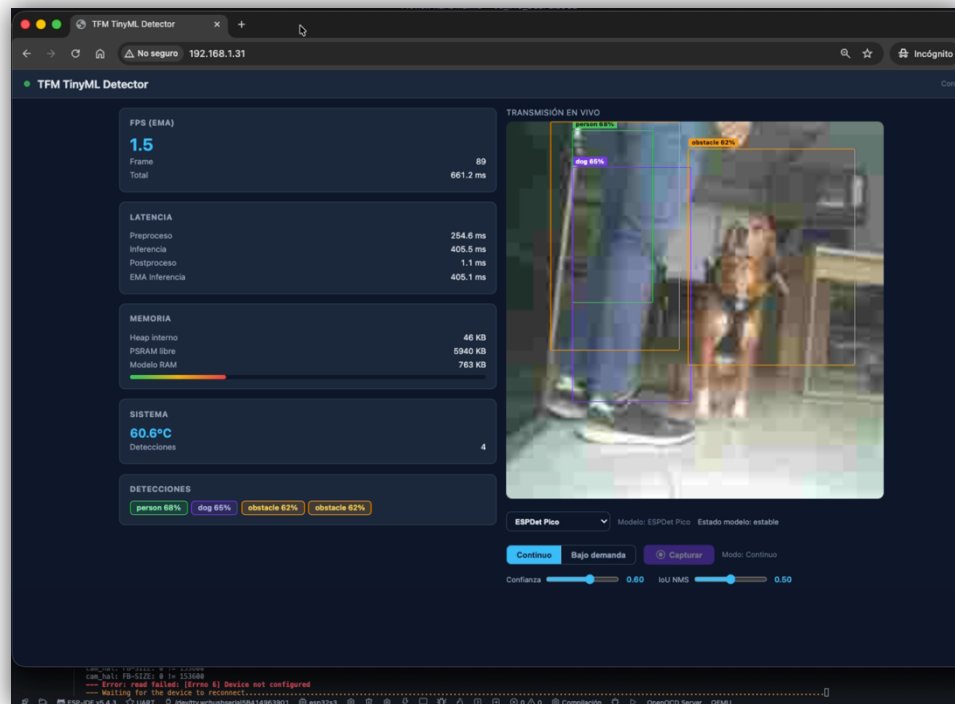


Figura 59. Pruebas de campo 5 Sistema de Detección de Objetos – Modelo ESPDet Pico T4



6.4.Síntesis

[

Tras la presentación de los datos, se realizará el análisis cualitativo:

- **Trade-off entre Precisión y Velocidad:** Discusión sobre si la ganancia en precisión de un modelo justifica el impacto en la latencia.
- **Fidelidad de la Cuantización:** Evaluación de cuánto rendimiento se perdió al pasar de FP32 a INT8 en el hardware real.
- **Cumplimiento de Objetivos:** Reflexión sobre si el sistema cumple con los requisitos técnicos para ser considerado el "sentido de la vista" de un robot móvil de bajo coste .

]